

GPU Acceleration for Deep Learning: Tensor Parallelism, Memory Movement, and Hardware Scale

Esther Toobian

May 26, 2026

Project Proposal

This project proposes a focused presentation on when and why GPUs accelerate deep-learning computations, and when the expected acceleration may fail. The topic builds directly on course material on tensors, batched linear layers, convolutional layers, computational graphs, autograd, and PyTorch modules. The presentation extends Fleuret’s discussion of GPU use in deep learning, where modern network size is identified as a major computational issue and GPUs are motivated by their suitability for signal processing and high-dimensional linear algebra [1].

The mathematical focus is the structure of neural-network tensor operations. A batched linear layer can be written as

$$Y = XW^T + \mathbf{1}b^T,$$

and a convolutional layer computes many local dot products over channels, kernels, and spatial positions. These operations expose large amounts of parallelism because many output entries can be computed independently or in parallel blocks. This explains why GPUs are effective for matrix multiplication, convolution, and the forward and backward passes of neural networks.

The project will also address limitations. GPU acceleration is not automatic. Small tensors, Python loops, insufficient batch size, memory bottlenecks, and frequent CPU-GPU transfers can reduce or eliminate the expected speedup. Fleuret emphasizes that tensors live on devices in PyTorch, operations are performed on the device where tensors reside, and moving data between CPU memory and GPU memory is comparatively slow [1]. Therefore, the presentation will include practical PyTorch guidance on using `device`, `tensor.to(device)`, `model.to(device)`, and keeping operands on compatible devices.

Experiments will compare local CPU, local laptop GPU, and, if access and scheduling permit, a PSU research-cluster GPU. The local GPU experiments will be run on a Lenovo Legion Pro 7 16IAX10H Type 83F5 with an NVIDIA GeForce RTX 5090 Laptop GPU. NVIDIA lists the RTX 5090 Laptop GPU with 10,496 CUDA cores and 24 GB of GDDR7 memory [4]. Planned experiments include CPU-versus-GPU matrix multiplication timing across input sizes, neural-network timing across batch sizes, vectorized tensor operations versus Python loops, and CPU-GPU transfer-overhead measurements. Multi-GPU computation will be treated as an optional extension using Fleuret’s discussion of `nn.DataParallel`. If cluster resources permit, results from a small multi-GPU experiment will be included.

The project is intended to be accessible to students with limited GPU experience while still providing depth through mathematical analysis, PyTorch device semantics, backend-library discussion, hardware comparison, and careful benchmarking methodology. The main goal is to develop a clear framework for reasoning about whether a deep-learning computation is likely to benefit from GPU acceleration before running it.

References

- [1] François Fleuret. *Deep Learning: 6.6 Using GPUs*. Course handout.
- [2] François Fleuret. *Deep Learning: 6.1 Benefits of Depth*. Course handout.
- [3] NVIDIA. *CUDA Platform for Accelerated Computing*. <https://developer.nvidia.com/cuda>
- [4] NVIDIA. *GeForce RTX 50 Series Laptop GPUs*. NVIDIA product documentation.
- [5] Lenovo. *Legion Pro 7 16IAX10H Platform Specifications*. Lenovo PSREF.
- [6] S. Shi, Q. Wang, P. Xu, and X. Chu. *Benchmarking State-of-the-Art Deep Learning Software Tools*. 2016.